

SAL: Multi-modal Verification of Replicated Data Types

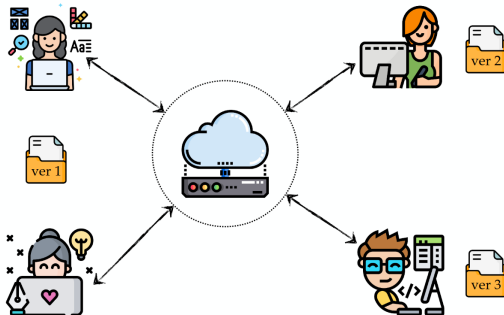
Pranav Ramesh Vimala Soundarapandian KC
Sivaramakrishnan

Indian Institute of Technology, Madras

April 22, 2026

Local-First Collaboration

- Users work offline, synchronize when connectivity returns
- Replicas evolve independently, must merge accurately
- Example: Shared documents, collaborative editors



Challenge: How do we ensure correctness when states diverge?

CRDTs and MRDTs: Building Blocks

CRDT Conflict-Free Replicated Data Type

- Deterministic two-way merge
- Requires tombstones

MRDT Mergeable Replicated Data Type

- Three-way merge (like Git)
- More compact representations (e.g., $O(1)$ counter state vs $\Omega(n)$ in CRDTs due to tombstones)

MERGE STRATEGIES

The merge mechanism (`git merge` and `git pull` commands) allows the backend merge strategies to be chosen with `-s` option. Some strategies can also take their own options, which can be passed by giving `-X<option>` arguments to `git merge` and/or `git pull`.

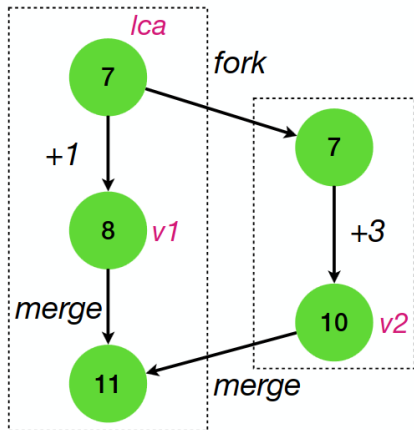
ort

This is the default merge strategy when pulling or merging one branch. This strategy can only resolve two heads using a `3-way merge algorithm`. When there is more than one common ancestor that can be used for 3-way merge, it creates a merged tree of the common ancestors

Three-way merge model in git (git-ort)

Example: Increment-Only Counter MRDT

Component	Definition
State	$\Sigma = \mathbb{N}, \sigma_0 = 0$
Initial	$\sigma_0 = 0$
Update	$\text{do}(\sigma, \dots, \text{inc}) = \sigma + 1$
Merge	$\text{merge}(\sigma_{lca}, \sigma_1, \sigma_2)$ $= \sigma_{lca} + (\sigma_1 - \sigma_{lca})$ $+ (\sigma_2 - \sigma_{lca})$
Conflicts	$\text{rc} = \emptyset$ (none)



Three-way merge after concurrent increments

Why Verification Matters

Designing correct replicated data types is challenging.

Errata

A few years after publication, we learned about some serious flaws in this paper:

- › The definition of non-interleaving in Section 2.1 cannot be guaranteed by any algorithm.
- › The algorithm presented in Section 3.1 does not guarantee convergence.

Please see the [Fugue paper](#) for more details.

RGA anomaly: The definition of non-interleaving in that paper cannot be satisfied by any algorithm

5.2 Buggy MRDT Implementation in [23]

We now present some details of one of the buggy MRDTs, Enable-wins flag, that we discovered using our framework in the work by Soundarapandian et al. [23]. The state of the enable-wins flag MRDT consists of a pair: a counter and a flag. The counter tracks the number of the enable events, while the flag is set to true on an enable event. The desired specification for this flag is that it should be true when there is at least one enable event not visible to any disable event.

Enable-wins flag bug: Incorrect merge logic allows disabled state to incorrectly become enabled

Replication-Aware (RA) Linearizability

Definition

RA linearizability relates replica executions to a sequential explanation of updates, ensuring that:

- The state at any replica during any execution must be achievable by applying a sequence (or linearization) of the update operations visible to the replica.
- This linearization should obey the user-specified conflict resolution policy for concurrent operations, and the local replica order for non-concurrent operations.
- Our definition of RA-linearizability allows viewing the state of an MRDT replica as a sequence of update operations applied on the initial state, thus abstracting over the merge function and how it handles concurrent operations.

Neem reduced RA-linearizability to 24 verifiable conditions (VCs)

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails
- Failed VC provides little actionable feedback
- Relies on external solver $\rightarrow$ larger trusted computing base (TCB)
- Debugging requires manual proof-debugging workflow
- SMT proofs are brittle to small changes

A Multi-Modal Workflow for RA Linearizability in Lean

- 1 **Kernel-checkable automation** with proof reconstruction
- 2 **SMT-aided automation** when needed (fallback)
- 3 **AI-assisted interactive theorem proving** for remaining obligations
- 4 **Counterexample generation** and visualization

Key insight: Lean's small trusted kernel → proofs checked without external SMT

Key Design: Data Structures for Automation

Challenge: Lean's standard library optimizes for manual proofs, not automation

Custom Set and Map Interfaces

```
-- Set: Boolean-valued membership
@[simp]
abbrev set (a:Type) [DecidableEq a] := a -> Bool

@[simp, grind]
def union {a:Type} [DecidableEq a] (s1 s2 : set a) : set a :=
  (fun x => s1 x || s2 x)

@[simp, grind]
def intersection {a:Type} [DecidableEq a] (s1 s2: set a) : set a :=
  (fun x => s1 x && s2 x)

-- Map: Explicit domain
structure map (key:Type) [DecidableEq key] (value:Type) where
  (mappings: key -> value) (domain: set key)
```

Automation-friendly: All operations are directly computable!

The Enable-Wins Flag Bug

Specification

A shared boolean flag with concurrent enable/disable operations. When enable and disable are concurrent, **enable wins**.

Naive Implementation Bug:

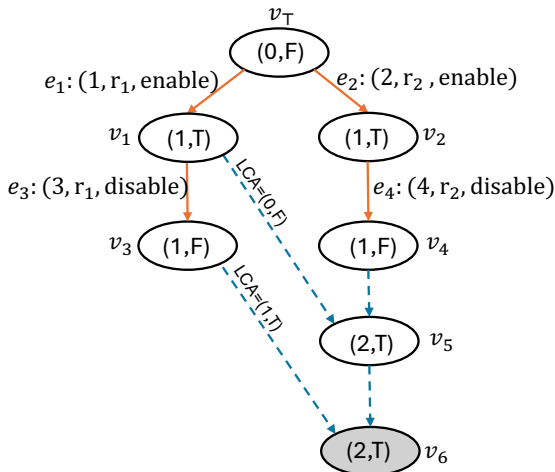
- Track concurrent enables with a counter
- Use counter to decide flag state after merge
- **Problem:** Enables followed by disables on each replica still show as enabled!

RA-Linearizability Verification Condition (VC)

```
theorem inter_right_lop (l: concrete_st) (a: concrete_st)
  (b: concrete_st) (o1: op_t) (ob: op_t) (ol: op_t) (o: op_t) :
  rc ob ol = rc_res.Fst_then_snd /\ get_rid ob != get_rid ol /\
  (~ (rc o ob = rc_res.Either) /\ (rc o ol = rc_res.Fst_then_snd)) /\
  distinct_ops ol ob /\ distinct_ops ol ol /\
  distinct_ops ol o /\ distinct_ops ob ol /\
  distinct_ops ob o /\ distinct_ops ol o /\
  get_rid o != get_rid ol /\
  eq (merge (do_ l ol) (do_ (do_ a ol) ol)
      (do_ (do_ b ob) ol))
      (do_ (merge (do_ l ol) (do_ a ol)
                  (do_ (do_ b ob) ol)) ol)
  ->
  eq (merge (do_ l ol) (do_ (do_ a ol) ol)
      (do_ (do_ (do_ b o) ob) ol))
      (do_ (merge (do_ l ol) (do_ a ol)
                  (do_ (do_ (do_ b o) ob) ol)) ol)
```

Conditions ensure interleaving property: operations commute correctly across merge boundaries

Concrete Counterexample: Enable-Wins Flag Execution



Automatic counterexample shows: both replicas disable the flag (v_3, v_4), yet merge produces $(2, \text{true})$ —flag is incorrectly enabled!

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

- 1 Express VCs as **decidable propositions**
- 2 Plausible generates random test inputs
- 3 Check whether VC holds for each input
- 4 When violation found: report the failing input

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

The Challenge

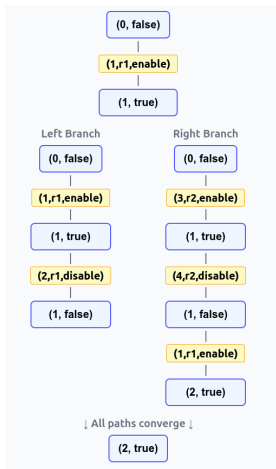
Raw counterexample is hard to understand—why does it fail?

Solution: ProofWidgets-based execution trace visualizer

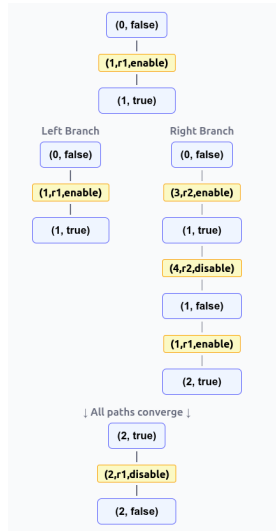
- Instrument do and merge operations
- Record intermediate states and operations
- Produce step-by-step execution trace
- Interactive visualization in Lean

Transforms opaque proof failures into tangible test cases

Enable-Wins Flag: Verification Conditions



LHS (merge result)



RHS (expected after linearization)

Stepping Back

SAL is more than tactics—it's a collaborative verification workflow:

- **Kernel automation:** Handles routine reasoning
- **SMT when needed:** For harder goals
- **AI assistance:** Proofs from Aristotle and Claude
- **Developer:** Makes informed decisions based on counterexample and proof strategies given by LLMs

AI-Assisted Verification: Real-World Integration

```
README: reflect current state (27 RDTs, all 24/24 VCs) over paper state == 95402d0
Peritext_CRDT: all 24 VCs closed via flat-set mark representation (19/24 -> 24/24) == 3243ae6
Peritext_CRDT: close 4 more VCs via per-component decomposition (15/24 -> 19/24) == 19acdf1
LWW_Map_CRDT: close remaining 4 VCs via Aristotle-generated intermediate lemmas == 33e5eb9
add Peritext_CRDT (first pass, 15/24 VCs closed, 9 TODO) == 7b435af
```

Integration commits

Commit 3243ae6

kayceesrk committed 5 hours ago

Peritext_CRDT: all 24 VCs closed via flat-set mark representation (19/24 -> 24/24)

The remaining 5 theorems (ind_lca_top, ind_left_top, ind_lca_top, ind_left_top, ind_right_top) were stuck at grind because the state represented marks as a nested 'map (OpId x Bool) (set MarkOp)' -- the inner 'set' as a map value forced grind to prove function equality via funext after unfolding.

Flattened to a top-level 'set AnchorAttachment' where 'AnchorAttachment := OpId x Bool x MarkOp'. This is the representation the paper's 'set alpha := alpha -> Bool' abstraction is designed for: top-level set membership is Boolean and grind can discharge it directly. With the flat representation, the per-component decomposition pattern ('refine <ab, ab, ab> <> intro k <> simp +decide ['] <> grind') closes all 5 previously-sorried theorems without SMT assistance.

Same pattern as Grow_Only_Set_CRDT and OR_Set_CRDT, which also use top-level 'set' rather than map-of-sets.

Peritext verification

What's verified

The suite currently contains **27 RDTs** — 18 CRDTs and 9 MRDTs — with all 24 RA-linearizability VCs closed on every one (648 kernel-verified VCs in total). Everything is checked on Lean v4.20.0 against the chore-bump-lean-4.28 branch of a [Blaster fork](#).

Verification status

Practical deployment of AI-assisted theorem proving in the SAL framework

Results Table

Table: Number of VCs discharged by the SAL tactic (total VCs = 24 per RDT). DG refers to dsimp + grind, and LB refers to lean – blaster.

RDT	DG	LB	ITP (Aristotle)	Time (s)
Increment-only counter MRDT	24	0	0	0.7
PN-counter MRDT	24	0	0	2.7
OR-set MRDT	3	21	0	17.3
Enable-wins flag MRDT	9	14	0	34.2
Efficient OR-set MRDT	2	22	0	14.4
Grows-only set MRDT	24	0	0	0.3
Grows-only map MRDT	22	0	2	25.8
Replicated growable array MRDT	15	9	0	3.9
Multi-valued register MRDT	24	0	0	0.3
Increment-only counter CRDT	24	0	0	1.5
PN-counter CRDT	16	2	6	99.5
Multi-valued register CRDT	24	0	0	0.7
OR-set CRDT	4	19	1	34.7

Thank you!

Questions?